

CSE 208: Data Structures and Algorithms II

Assignment on Maximum Flow

Deadline: 31 July 2026 10:30 PM

Introduction

In maximum flow problem, given a directed graph with a source node s and a sink node t , with each edge having a capacity that represents the maximum amount of flow that can be sent through it, the goal is to find the maximum amount of flow that can be sent from s to t , while respecting the capacity constraints on the edges.

This assignment has two parts:

- Implementing the Edmonds–Karp algorithm, a BFS-based version of Ford–Fulkerson for maximum flow
- Applying flow and matching ideas to solve a real-world optimization problem: Airline Fleet Scheduling

Part I: Maximum Flow and Edmonds–Karp Algorithm

Maximum Flow Problem

Given a directed graph $G = (V, E)$, where each edge has a capacity, the goal is to find the maximum flow from a designated source s to a designated sink t such that:

- The flow on each edge does not exceed its capacity
- Flow conservation holds at every node except s and t

In other words, we want to push as much flow as possible from the source to the sink without violating edge capacities.

Edmonds–Karp Algorithm

The Edmonds–Karp algorithm is a specific implementation of Ford–Fulkerson that uses Breadth-First Search (BFS) to find augmenting paths in the residual graph.

Algorithm Steps

1. Initialize total flow to 0
2. While there exists a path from s to t in the residual graph:
 - Use BFS to find an augmenting path
 - Find the minimum residual capacity along that path
 - Augment the flow along the path by that amount
 - Update the residual graph
3. Return the total flow

Test Cases

For this part, the input format is as follows:

- The first line contains two integers N and M
- N denotes the total number of nodes in the flow graph
- Node 0 is the source and node N-1 is the sink
- M is the number of directed edges
- The next M lines each contain three integers: u v w, where w is the capacity of the edge from u to v

Your program should output the maximum flow of the graph.

Test Case 1

Input

```
6 8
0 1 10
0 2 10
1 2 4
1 3 2
3 4 9
2 5 10
4 2 6
4 5 10
```

Expected Output

12

Test Case 2

Input

5 5

0 1 8

0 2 5

1 2 3

2 3 4

3 4 10

Expected Output

4

Part II: Airline Fleet Scheduling Problem

Problem Description

An airline operates multiple flights during a day. Each flight has:

- Departure city
- Arrival city
- Departure time
- Arrival time

Your task is to determine:

1. The minimum number of aircraft required to cover all flights
2. The sequence of flights assigned to each aircraft

Constraint

A single aircraft can perform flight B after flight A if:

1. The arrival city of A is the same as the departure city of B
2. A's arrival time plus 3 hours turnaround time is less than or equal to B's departure time

This means that after completing a flight, an aircraft needs 3 hours before it can take off again from the same city.

Solution Idea

One possible way of assigning aircrafts to flights is to assign one aircraft to each flight. If there are K flights, this assignment will require K aircrafts. Every time an aircraft can be scheduled to fly flight F_i followed by F_j , the two flights can be merged into one route, reducing the number of required aircrafts by 1. To find the minimum required number of aircrafts, we need to maximize the number of such merged flights completed by each aircraft. This problem reduces to maximum bipartite matching, which can be solved by the Ford-Fulkerson algorithm.

Graph Modeling (DAG)

We model the problem as a directed graph:

- Each flight is a node
- Add a directed edge $A \rightarrow B$ if one aircraft can perform flight A and then flight B

Because time always moves forward, this graph is a Directed Acyclic Graph (DAG).

However, the goal is not simply to find paths in this DAG. The goal is to cover all flights using the minimum number of aircraft, where each aircraft follows one valid sequence of flights.

Reduction to Minimum Path Cover

We want to cover all flights using the minimum number of vertex-disjoint paths.

Theorem

Minimum Path Cover} = $N - \text{Maximum Matching}$

where N is the number of flights.

Intuition

Each matched edge connects two flights into the same aircraft route. Every such connection reduces the number of separate paths by 1. Therefore, if we maximize the number of valid connections, we minimize the number of aircraft required.

Conversion to Bipartite Graph

To apply the theorem, we convert the DAG into a bipartite graph:

- Split each flight node v into:
 - v_{out} on the left side
 - v_{in} on the right side
- For every feasible connection $A \rightarrow B$, add an edge:
 - $A_{\text{out}} \rightarrow B_{\text{in}}$

This bipartite graph represents all possible valid flight-to-flight transitions.

Conversion to Flow Network

Now we convert the bipartite graph into a flow network:

- Add an edge from the source to every left-side node with capacity 1
- Add an edge from every left-side node to its feasible right-side neighbors with capacity 1
- Add an edge from every right-side node to the sink with capacity 1

Then:

- Maximum flow = Maximum matching
- Minimum aircraft = Number of flights - Maximum matching

This is the point where Edmonds–Karp can be used directly.

Example

Flights

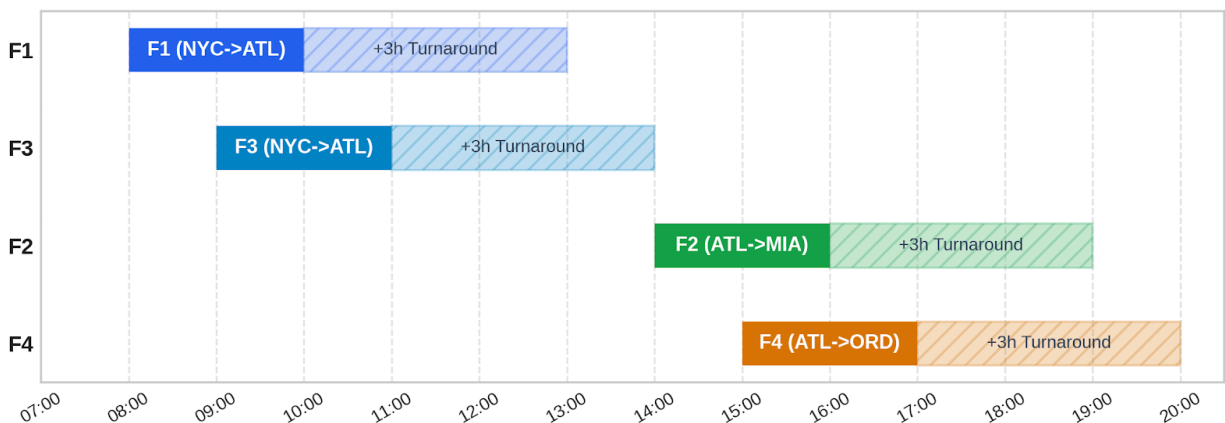
Flight	From	To	Departure Time	Arrival Time
F1	NYC	ATL	08:00	10:00
F2	ATL	MIA	14:00	16:00
F3	NYC	ATL	09:00	11:00
F4	ATL	ORD	15:00	17:00

Add Turnaround Time

Turnaround time = 3 hours

- F1 arrives at 10:00, so the earliest next departure is 13:00
- F3 arrives at 11:00, so the earliest next departure is 14:00

Step 1: Flight Timelines with 3-Hour Turnaround Buffers



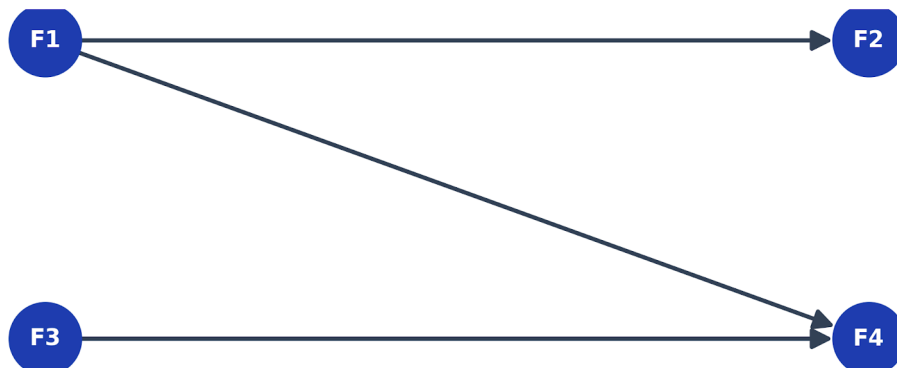
Step 2: Find Feasible Connections

A connection is feasible if the arrival city of the first flight matches the departure city of the second flight and the second flight departs at least 3 hours later.

Feasible connections:

- $F1 \rightarrow F2$ ($14:00 \geq 13:00$)
- $F1 \rightarrow F4$ ($15:00 \geq 13:00$)
- $F3 \rightarrow F4$ ($15:00 \geq 14:00$)

Step 2: Directed Acyclic Graph (DAG) of Feasible Connections



Step 3: Build the Bipartite Graph

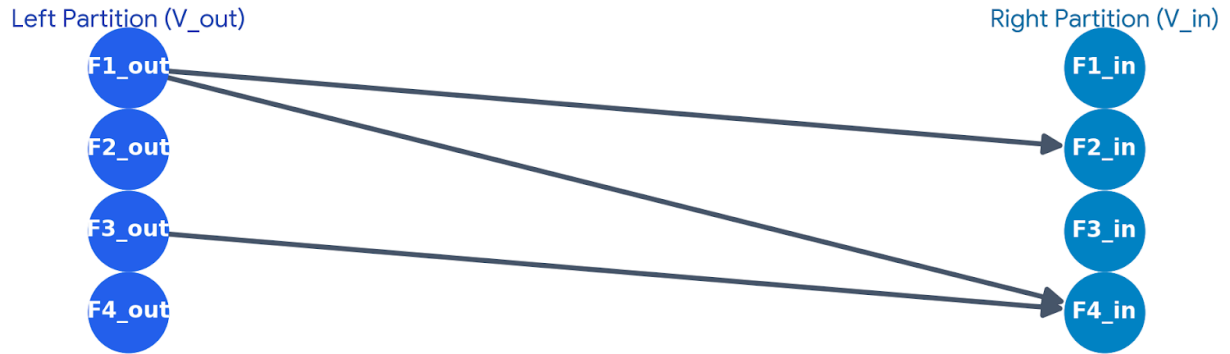
Left side: F1_out, F2_out, F3_out, F4_out

Right side: F1_in, F2_in, F3_in, F4_in

Edges:

- $F1_out \rightarrow F2_in$
- $F1_out \rightarrow F4_in$
- $F3_out \rightarrow F4_in$

Step 3: Bipartite Graph Split ($V_{out} \rightarrow V_{in}$)



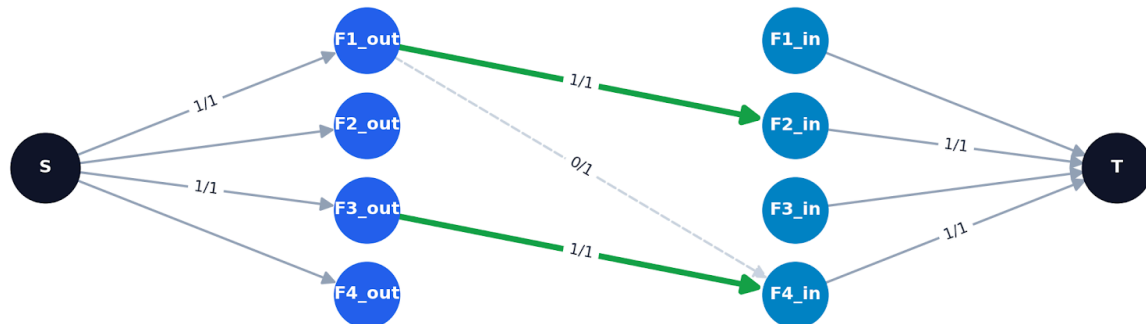
Step 4: Maximum Matching

One possible maximum matching is:

- $F1 \rightarrow F2$
- $F3 \rightarrow F4$

Matching size = 2

Step 4: Flow Network & Max Matching (Green = Flow Executed, Max Flow = 2)



Step 5: Minimum Aircraft

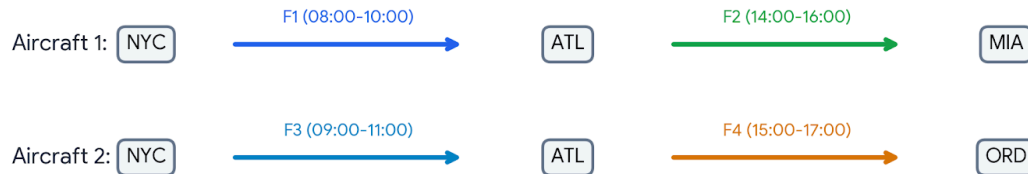
$$4 - 2 = 2$$

So, the minimum number of aircraft required is 2.

Step 6: Aircraft Routes

- Aircraft 1: F1 → F2
- Aircraft 2: F3 → F4

Step 5 & 6: Minimum Aircraft Cover & Final Assigned Routes (2 Aircraft Total)



Test Cases (Airline Scheduling)

For this part, the input format is as follows:

- The first line contains an integer F, the number of flights
- The next F lines each contain:
 - Flight ID
 - Departure city
 - Arrival city
 - Departure time
 - Arrival time

Times are given in 24-hour format as HH :MM.

Your program should print:

1. The minimum number of aircraft required
2. The route of flights assigned to each aircraft

If multiple optimal answers exist, any valid optimal solution will be accepted.

Test Case 1

Input

5

F1 A B 06:00 08:00

F2 B C 11:30 13:30

F3 A B 07:00 09:00

F4 B D 12:30 14:30

F5 D E 18:00 20:00

Feasible Connections

- F1 → F2
- F1 → F4
- F3 → F4
- F4 → F5

Maximum Matching

Maximum matching size = 3

Minimum Aircraft

$$5 - 3 = 2$$

Expected Output

Number of Aircraft: 2

Aircraft 1: F1 -> F2

Aircraft 2: F3 -> F4 -> F5

Test Case 2

Input

6

F1 X Y 05:00 07:00

F2 Y Z 10:30 12:30

F3 Z X 16:00 18:00

F4 X Y 08:00 10:00

F5 Y Z 14:00 16:00

F6 Z W 19:30 21:30

Feasible Connections

- F1 → F2
- F2 → F3
- F4 → F5
- F5 → F6

Maximum Matching

Maximum matching size = 4

Minimum Aircraft

$$6 - 4 = 2$$

Expected Output

Number of Aircraft: 2

Aircraft 1: F1 -> F2 -> F3

Aircraft 2: F4 -> F5 -> F6

Mark Distribution

Correct implementation of Edmonds-Karp algorithm	40
Correctly reading and converting flight information to a DAG	10
Converting DAG to bipartite graph	10

Compute maximum matching and find routes	20
Print output with proper formatting	10
Viva	10

7. Submission Guidelines

If you have a single solution file, rename it with your ID. In case of multiple files, put all your files in a directory named with your ID, zip it, and submit the zipped file to moodle. If your ID is 2405XXX, you should submit a 2405XXX.cpp or 2405XXX.zip file.

Please do not copy. Copying another student's work, or submitting code copied from websites or generative AI tools may result in penalties according to the course policy.